Chapter 30: OnPush Deep Dive

1. Overview

`ChangeDetectionStrategy.OnPush` is the single highest-leverage performance switch in an Angular application. Volume 2 introduced it as "skip components whose inputs haven't changed." That description is correct but dangerously incomplete for interview purposes and for production debugging. The real behavior is governed by four precise triggers that mark a component's `LView` dirty, a reference-equality check (not deep equality, not `isEqual`), and an interaction model with zones, signals, and third-party libraries that trips up even experienced Angular engineers.

This chapter treats OnPush as a contract you must actively uphold through immutability, rather than a flag you flip and forget. We cover:

- The exact four conditions that cause Angular to run change detection on an OnPush component.
- Why reference equality (`===`) is the only check performed on `@Input()` bindings, and what that means for objects, arrays, and nested structures.
- Immutable update patterns that make OnPush safe: spread, `structuredClone`, `Object.freeze`, and immutable libraries (Immer, Immutable.js).
- How the async pipe complements OnPush by calling `markForCheck()` internally on every emission.
- How Signals bypass the whole reference-equality problem via fine-grained dependency tracking — the biggest paradigm shift since OnPush was introduced.
- What happens when third-party code (jQuery plugins, D3, non-Angular widgets) mutates the DOM or your data without Angular's knowledge.
- Testing OnPush components correctly in `TestBed`, including the common trap of forgetting `detectChanges()` calls or fixture defaults.
- A concrete, incremental strategy for migrating a large Default-strategy application to OnPush without a "big bang" rewrite.

2. Core Concepts

2.1 What "dirty" actually means

Every component instance backed by Ivy has an `LView` — an array-based internal data structure holding the component's bindings, DOM references, and a `FLAGS` bit field. One of those flags is `LViewFlags.Dirty` (conceptually; the actual check is `CheckAlways` vs `OnPush` semantics combined with `Dirty` / `RefreshView` flags). When Angular's change detection tick walks the component tree, it visits every `CheckAlways` (Default strategy) component unconditionally, but for `OnPush` components it only descends into and refreshes the view if that view has been explicitly marked dirty (or is a `RefreshView` due to a marked ancestor path).

"Marking dirty" is the operative act. Angular does **not** periodically re-check OnPush components hoping something changed — it needs to be told. There are exactly four ways that can happen.

2.2 The four triggers, precisely

Trigger 1 — Input reference change.
When a parent template re-renders and produces a new value for one of the child's `@Input()`-bound bindings, Angular compares the new value to the previously stored value using `Object.is` (SameValueZero, effectively `===` semantics for objects/arrays, with the small non-observable difference that `NaN === NaN` is `false` but `Object.is(NaN, NaN)` is `true`). If the reference differs, the binding is updated **and** the component is marked dirty (`markForCheck` is invoked internally via `bindingUpdated` / `markViewDirty` in Ivy's instruction set). Primitives compare by value, since `1 === 1` is definitionally true; objects and arrays compare only by pointer.

```
// Parent template: <app-child [items]="items"></app-child>
this.items.push(newItem);      // ❌ same array reference — child NOT marked dirty
this.items = [...this.items, newItem]; // ✅ new reference — child marked dirty
```

Trigger 2 — An event originates from the component or one of its descendants.
Whenever a native DOM event (click, input, keyup, etc.) bound anywhere inside an OnPush component's template (including in child components nested inside it) fires, Angular runs change detection for the **entire path from the root to that component**, not just that component. This is because Angular can't know in advance whether the event handler mutated some shared state that other branches of the tree depend on, so by default (with zone.js) a full tree walk happens on any event — but critically, **OnPush components not on the path from root to the event source, and not otherwise dirty, are still skipped** during that walk if they are clean. The component where the event originated, and all of its ancestors up to the root, get their dirty/refresh flags implicitly satisfied because they are on the "notify path." This is often summarized as "OnPush components refresh on events dispatched from within their own template."

```
@Component({
  selector: 'app-counter',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<button (click)="increment()">{{ count }}</button>`
})
export class CounterComponent {
  count = 0;
  increment() { this.count++; } // this component is marked dirty because the
                                // click event originated inside its own template
}
```

Trigger 3 — Async pipe emission.
The `AsyncPipe` holds an internal reference to the `ChangeDetectorRef` of the component whose template it's used in. Every time the underlying `Observable` emits (or the `Promise` resolves), the pipe calls `this._ref.markForCheck()` before returning the new value to the template. This is why `| async` is the idiomatic OnPush partner — it manually performs Trigger 4 on every emission automatically, without you writing any code.

Trigger 4 — Explicit `markForCheck()` (or a signal read that changed).
You (or a library like `AsyncPipe`, the Router, or the `HttpClient` when used with signals) can call `ChangeDetectorRef.markForCheck()` directly to say "I know something changed, please refresh this view and all ancestors up to the root on the next tick." Additionally, in modern Angular (v16+), reading a signal in a template automatically registers that template as a "consumer" in the reactive graph; when the signal's value

changes, Angular schedules a refresh of exactly that component's view — this is functionally a more precise version of `markForCheck()` that doesn't require the zone or an explicit call.

> **The exhaustive list, restated:**
>
> 1. `@Input()` reference changed (checked via `Object.is`).
> 2. DOM event fired from within the component's own view (including child view events, which bubble the "dirty" implications up the ancestor chain).
> 3. `AsyncPipe` received a new emission and called `markForCheck()`.
> 4. Manual `markForCheck()` call, or (Angular 16+) a signal read in the template changed value and Angular's reactive graph scheduled a targeted refresh.

If none of these four things happen, an OnPush component's view is **never** re-rendered, no matter how many times its internal fields mutate.

2.3 Reference equality is the entire contract

Angular does not do deep comparison, `JSON.stringify` comparison, or structural diffing on `@Input()` bindings — ever. This is intentional: deep comparison would be O(n) on every change-detection cycle for every binding, defeating the performance purpose of OnPush. The contract you must uphold is: **if the semantic content of a value changes, its reference must also change.** This pushes the responsibility for immutability onto you, the developer, which is exactly what Section 3 (immutable update patterns) addresses.

2.4 Local mutations still trigger re-render of the *same* component

A subtlety often missed: Trigger 2 means that even though OnPush skips re-checking a component when its **inputs** are unchanged, the component itself is still checked normally (like a Default component) whenever an event handler runs inside it. Inside that check, if you mutate `this.someField = x` directly (not via input), the template will pick it up correctly because the component's own view was refreshed. OnPush restricts *ancestor-triggered* re-checks caused by unrelated sibling events elsewhere in the tree — it does not turn off change detection for the component's own local state when a local event fires.

3. Code Examples

3.1 Immutable update patterns — objects

```
interface UserProfile {
  id: number;
  name: string;
  address: { city: string; zip: string };
  tags: string[];
}

@Component({
  selector: 'app-profile-editor',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `<app-profile-card [profile]="profile"></app-profile-card>`
})
export class ProfileEditorComponent {
  profile: UserProfile = {
```

```
    id: 1,
    name: 'Ava',
    address: { city: 'Austin', zip: '78701' },
    tags: ['admin']
  };

  // ❌ WRONG — mutates in place, same top-level reference, child never re-renders
  renameWrong(newName: string) {
    this.profile.name = newName;
  }

  // ✅ Shallow spread — safe when only a top-level field changes
  renameCorrect(newName: string) {
    this.profile = { ...this.profile, name: newName };
  }

  // ✅ Nested update requires spreading every level on the path that changed
  updateCityCorrect(city: string) {
    this.profile = {
      ...this.profile,
      address: { ...this.profile.address, city }
    };
  }

  // ❌ WRONG — nested object mutated, only inner reference changes,
  // but if `address` itself is bound as a separate @Input() to a grandchild,
  // that grandchild WILL be missed because `profile` (outer) reference is stale
  // for anything relying on it, while address's mutation is invisible entirely.
  updateCityWrong(city: string) {
    this.profile.address.city = city; // top-level `profile` ref unchanged
  }

  // ✅ structuredClone — convenient for deep, one-shot clones (modern browsers/Node 17+)
  addTagWithClone(tag: string) {
    const cloned = structuredClone(this.profile);
    cloned.tags.push(tag);
    this.profile = cloned;
  }

  // ✅ Arrays — never push/splice/sort in place on OnPush-bound arrays
  addTagCorrect(tag: string) {
    this.profile = { ...this.profile, tags: [...this.profile.tags, tag] };
  }
  removeTagCorrect(tag: string) {
    this.profile = {
      ...this.profile,
      tags: this.profile.tags.filter(t => t !== tag)
    };
  }
}
```

3.2 Immutable updates with Immer (production pattern for deep state)

```
import { produce } from 'immer';

interface AppState {
  cart: { items: { id: string; qty: number }[]; total: number };
}

export class CartStore {
  private state: AppState = { cart: { items: [], total: 0 } };

  // Immer lets you "mutate" a draft; it produces a new, structurally-shared
  // immutable object under the hood — perfect for OnPush + deeply nested state.
  incrementQty(itemId: string) {
    this.state = produce(this.state, draft => {
      const item = draft.cart.items.find(i => i.id === itemId);
      if (item) item.qty++;
    });
  }
}
```

3.3 OnPush + async pipe synergy

```
@Component({
  selector: 'app-order-list',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <ul>
      <li *ngFor="let order of orders$ | async; trackBy: trackById">
        {{ order.id }} — {{ order.status }}
      </li>
    </ul>
  `
})
export class OrderListComponent {
  orders$ = this.orderService.orders$; // Observable<Order[]>

  constructor(private orderService: OrderService) {}

  trackById(_: number, order: Order) { return order.id; }
}
```

No `markForCheck()` is ever written by hand here. Every emission on `orders$` triggers the `AsyncPipe`'s internal `markForCheck()`, satisfying Trigger 3. Combined with `trackBy`, this avoids DOM churn even when new array references arrive on every emission.

3.4 OnPush + Signals — automatic fine-grained updates

```
@Component({
```

```
  selector: 'app-cart-summary',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <p>Items: {{ itemCount() }}</p>
    <p>Total: {{ total() }}</p>
  `
})
export class CartSummaryComponent {
  private cartService = inject(CartService);

  // signals — no async pipe, no manual markForCheck, no input reference dance
  itemCount = this.cartService.itemCount; // Signal<number>
  total = computed(() => this.cartService.items().reduce((s, i) => s + i.price * i.qty,
0));
}

@Injectable({ providedIn: 'root' })
export class CartService {
  items = signal<{ id: string; price: number; qty: number }[]>([]);
  itemCount = computed(() => this.items().length);

  addItem(item: { id: string; price: number; qty: number }) {
    this.items.update(current => [...current, item]); // still immutable update —
    // signals don't remove the need for immutability on the *stored value*,
    // they remove the need for manual markForCheck / input-reference plumbing.
  }
}
```

Note the subtlety: even with signals, `items.update()` should still produce a new array so that `computed()` and any `effect()`/template consumers reliably see a changed value via the signal's own equality check (`Object.is` by default). Signals don't grant you license to mutate arbitrarily — they replace *how Angular is notified*, not the requirement that state transitions be observable.

3.5 Step-by-step: migrating a Default component to OnPush

Starting point — a Default-strategy component with silent mutation-based updates:

```
// BEFORE
@Component({
  selector: 'app-todo-list',
  template: `
    <div *ngFor="let todo of todos">
      <input type="checkbox" [checked]="todo.done" (change)="toggle(todo)">
      {{ todo.text }}
    </div>
    <button (click)="addFromParentEvent()">Add</button>
  `
})
export class TodoListComponent {
  @Input() todos: Todo[] = [];

  toggle(todo: Todo) {
```

```
    todo.done = !todo.done; // mutates in place — fine under Default CD
  }
}
```

Step 1 — Add the strategy flag and immediately audit every mutation site.

```
@Component({
  selector: 'app-todo-list',
  changeDetection: ChangeDetectionStrategy.OnPush, // added
  template: `...` // unchanged for now
})
```

Step 2 — Fix in-place mutations to produce new references.

```
// AFTER
export class TodoListComponent {
  @Input() todos: Todo[] = [];
  @Output() todosChange = new EventEmitter<Todo[]>();

  toggle(todo: Todo) {
    // Because this runs from a (change) event bound in this component's own
    // template, Trigger 2 fires and this component IS checked regardless.
    // But if the parent also needs updated data (e.g. it owns `todos` and
    // passes it back down elsewhere), the array must be replaced immutably
    // and pushed back up so sibling/parent OnPush views see it too.
    const updated = this.todos.map(t =>
      t === todo ? { ...t, done: !t.done } : t
    );
    this.todos = updated;
    this.todosChange.emit(updated);
  }
}
```

Step 3 — Update the parent to replace, not mutate, the array it owns.

```
@Component({
  selector: 'app-todo-page',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <app-todo-list [todos]="todos" (todosChange)="onTodosChange($event)"></app-todo-list>
  `
})
export class TodoPageComponent {
  todos: Todo[] = [];

  onTodosChange(updated: Todo[]) {
    this.todos = updated; // new reference propagates to any other OnPush consumers
  }
}
```

Step 4 — Verify every `@Input()` binding source across the tree. Grep for direct property mutation (`.push(`, `.splice(`, `foo.bar =`) on anything reachable from an `@Input()`-bound object, and replace with spread/`map`/`filter`/Immer as appropriate.

Step 5 — Run with `ngDevMode` / Angular DevTools' change-detection profiler to confirm the component no longer re-renders on unrelated sibling events, and does re-render when its own inputs legitimately change.

4. Internal Working

4.1 The LView flags and the tree walk

Ivy stores per-view state in an `LView`, a JS array where a fixed low index (`LView[FLAGS]` conceptually) holds a bitmask including whether the view uses `CheckAlways` (Default) or `OnPush` semantics, and whether it is currently `Dirty`/needs a `RefreshView`. During `tick()`, Angular's `refreshView`/`detectChangesInternal` walks the component tree top-down. For each `LView`:

- If the view's strategy is `CheckAlways`, it is refreshed unconditionally — this is Default strategy behavior, and it's why a single unrelated event anywhere in a Default-heavy tree can re-run bindings across the whole app.
- If the view's strategy is `OnPush`, Angular checks the `Dirty` flag (and whether it's flagged for refresh due to being on the "notify path" from an event or a `markForCheck()` call). If clean, the walk **prunes that subtree entirely** — none of its children are visited, regardless of their own strategy, because the assumption is nothing under a clean OnPush root could have legitimately changed without the root itself being notified first (input changes propagate to children as new bindings during the parent's own check, and events/async/markForCheck have their own escalation described below).

4.2 How `markForCheck()` escalates

`markForCheck()` does not just flag the current component — it walks **up** the ancestor chain, marking every ancestor `LView` as needing a refresh path (without marking them fully "dirty" in the CheckAlways sense, but enough that the tree walk won't prune past them). This is essential: if a deeply nested OnPush component calls `markForCheck()`, but its OnPush parent were still considered "clean" and pruned, the walk would never reach the descendant that requested the check. Escalation to the root guarantees the path down to the requesting component survives pruning, while sibling subtrees that are still clean remain pruned.

4.3 Why input-reference marking works this way

When a parent's own view refresh runs and it re-evaluates the binding expression `[items]="items"`, Angular's generated instruction (`ɵɵproperty`) calls `bindingUpdated(lView, index, value)` under the hood, which does the `Object.is` comparison against the previously stored value at that binding slot. If different, it updates the slot and calls the equivalent of `markViewDirty` on the **child's** `LView`. This is the mechanism, at the instruction level, for Trigger 1 — it's not magic, it's a generated function call comparing two values by reference, exactly like you'd write by hand.

4.4 Why Signals sidestep the whole problem

Signals don't rely on the tree-walk-and-flag mechanism at all for their core notification — they rely on a separate, fine-grained reactive graph (the same primitive that powers `computed()` and `effect()`). When a signal is read inside a template, Ivy's template compiler generates a "consumer" registration tying that specific template's rendering function to that signal as a dependency (via `SIGNAL` node "producer/consumer" tracking, conceptually similar to how MobX or Vue's reactivity works). When the signal's value is `set()` / `update()`ed, only the consumers that actually read it are scheduled for a targeted, synchronous-on-next-tick refresh — bypassing the need for `@Input()` reference checks, `markForCheck()`, or even zone.js's event patching entirely. This is why Angular's zoneless mode (stable as of Angular 18+ experimental, more mature in later versions) is only viable *because* signals provide this independent notification channel — without them, removing zone.js would mean losing Trigger 2 (event-based CD) with no replacement.

In short: OnPush + manual inputs asks "did the reference change?" reactively-but-coarsely at the component boundary. Signals ask "did the specific reactive node this template depends on change?" at arbitrary granularity, inside or across component boundaries, without needing a parent to "pass down" a new object reference at all.

5. Edge Cases & Gotchas

5.1 Deeply nested mutation is invisible, and it's easy to convince yourself it "worked."
If `profile.address.city = 'Dallas'` mutates a nested object and some *other*, unrelated event elsewhere in the app happens to trigger a full CD cycle before you look, you might see the DOM update and wrongly conclude OnPush "picked up the mutation." It didn't — a *different* trigger caused a refresh that happened to reflect the already-mutated-in-memory value. This is the single most common source of "OnPush works... sometimes" bug reports.

5.2 `ChangeDetectorRef.detectChanges()` vs `markForCheck()`.
`detectChanges()` synchronously runs change detection **for this component and its descendants right now**, regardless of dirty flags — it does not walk up to ancestors and does not respect the OnPush skip logic on the way down from itself. `markForCheck()` instead flags the component (and escalates up to the root) so that the **next** scheduled tick includes it. Calling `detectChanges()` repeatedly in a hot path (e.g., inside a scroll handler) can defeat the entire purpose of OnPush by forcing synchronous, unthrottled recomputation. Prefer `markForCheck()` for "something changed, please refresh on the normal cycle," and reserve `detectChanges()` for imperative contexts like tests or manual bootstrapping outside Angular's normal flow (e.g., inside a `requestAnimationFrame` callback where you explicitly want a synchronous, immediate DOM sync).

5.3 `ChangeDetectorRef.detach()` + manual `detectChanges()` for high-frequency updates.
A common advanced pattern: `detach()` a component from the tick cycle entirely, then call `detectChanges()` yourself on a throttled interval (e.g., a live-updating chart). This is orthogonal to OnPush vs Default — `detach()` removes the view from automatic ticking regardless of strategy — but it's frequently combined with OnPush for maximal control.

5.4 Third-party DOM/data mutation Angular never sees.
A jQuery plugin, a non-Angular chart library, or a Web Component that mutates the DOM directly outside Angular's rendering path can desync the DOM from Angular's internal view state. This isn't strictly an OnPush-only problem (Default has the same blind spot for direct DOM writes), but it's more commonly *surfaced* under OnPush because engineers assume "OnPush isn't updating" when actually the real issue is the data bound to Angular templates was never touched — only the DOM was, by a foreign script. The fix is always the same: route all data mutations that should reflect in Angular templates through Angular-owned state (signals,

`@Input()`, or services), and treat third-party DOM writes as opaque and non-authoritative for anything Angular also renders.

5.5 `ngOnChanges` still fires correctly under OnPush — it's still input-reference-triggered.
`ngOnChanges` is not disabled by OnPush; it fires whenever Trigger 1 fires (new input reference), same as under Default. The difference is purely about whether the *template* gets re-rendered on unrelated ancestor/sibling activity, not whether lifecycle hooks tied to input changes still run.

5.6 `@ViewChild` / `@ContentChild` static queries and OnPush timing.
Because OnPush components may skip cycles, code that assumes "my view child is up to date because a tick just happened somewhere in the app" can read stale references if the OnPush component itself was pruned from that particular tick. Always trigger off the component's own lifecycle/CD, not assumptions about global tick frequency.

5.7 Testing pitfall — forgetting `fixture.detectChanges()` after mutating a test's bound `@Input()`.
In `TestBed`-driven tests, `fixture.detectChanges()` runs `ngOnInit` and initial binding on first call ,but subsequent input changes made directly on the component instance (`fixture.componentInstance.foo = ...`, bypassing a real parent template binding) do **not** automatically re-trigger CD for OnPush components the way a real parent re-render would — you must call `fixture.detectChanges()` again explicitly. A related trap: setting a new object but forgetting the object must be a *new reference* even in the test, otherwise the assertion can pass by accident (because the component's old-in-memory field already held the mutated value) while masking a real production bug.

5.8 Router-outlet-hosted OnPush components and navigation.
Router-driven component swaps go through their own creation lifecycle, so newly routed OnPush components render correctly on creation; the gotcha is with **resolvers/route data reused across navigations to the same component instance** (e.g., a component reused via a custom `RouteReuseStrategy`) — if only route params/data change but the component isn't destroyed/recreated, you must explicitly re-fetch and mark for check, since the component's own fields silently updating from a subscription won't retrigger CD unless one of the four triggers applies (typically solved by using the async pipe on route data observables).

6. Interview Questions & Answers

Q1. What does `ChangeDetectionStrategy.OnPush` actually change about a component?
A: It changes when Angular decides to re-run change detection (re-evaluate template bindings) for that component. Instead of checking on every tick (Default/`CheckAlways`), Angular only checks it when the component is marked dirty via one of four triggers: an `@Input()` reference change, a DOM event originating within the component's own view, an `AsyncPipe` emission, or an explicit `markForCheck()` call / signal read change. It does not change how bindings are evaluated once a check happens — only whether the check happens at all.

Q2. Why does mutating an array bound via `@Input()` not update an OnPush child, but replacing it does?
A: Angular compares the new binding value to the previous one using reference equality (`Object.is`), not deep equality. `array.push(x)` keeps the same array reference, so the comparison sees "no change" and never marks the child dirty. `array = [...array, x]` produces a new reference, the comparison detects a difference, updates the stored binding value, and marks the child's view dirty so it gets checked on the next tick.
Interviewer intent: This checks whether the candidate actually understands the mechanism (reference equality) rather than reciting "OnPush needs immutability" as a memorized rule without knowing why.

Q3. List the exact conditions under which an OnPush component gets checked.
A: Four: (1) one of its `@Input()` bindings receives a new reference (primitives compare by value inherently); (2) a DOM event handler bound anywhere in its own template (or a descendant's) fires, which marks the check path from that component up to the root; (3) an `AsyncPipe` used in its template receives a new emission and internally calls `markForCheck()`; (4) code explicitly calls `ChangeDetectorRef.markForCheck()`, or, in modern Angular, a signal read in its template changes value, triggering a targeted reactive refresh.

Q4. If a button inside an OnPush component triggers a click handler that only mutates a field on a *sibling* component (via a shared service, not inputs), will the sibling re-render?
A: Not automatically. The click event marks the check path from the clicked component up to the root, but the sibling subtree is unrelated to that path; if the sibling is OnPush and none of its own four triggers fire, it stays pruned and won't reflect the mutated field. The shared service needs to expose the state via an `@Input()` (with a new reference), an `Observable` consumed with `| async`, a signal, or the sibling needs its own explicit `markForCheck()` call (e.g., via a shared `ChangeDetectorRef` reference, which is an anti-pattern) to actually re-render.
Interviewer intent: Tests whether the candidate falsely believes "any event anywhere causes global re-check under OnPush," a common half-truth from conflating zone.js's global event patching with per-component dirty-checking.

Q5. How does the `AsyncPipe` cooperate with OnPush internally?
A: `AsyncPipe` subscribes to the bound `Observable`/`Promise` and holds a reference to the host component's `ChangeDetectorRef`. On every new emission, before returning the value to the template, it calls `this._ref.markForCheck()`. This satisfies Trigger 4 automatically on every emission, which is why `| async` is considered the idiomatic, "batteries included" partner for OnPush — you get correct, minimal-latency updates without writing any manual CD-triggering code, and the pipe also handles unsubscription on destroy.

Q6. Explain how Signals avoid the reference-equality limitation that plagues plain `@Input()` + OnPush.
A: Signals don't use the input-binding reference-check mechanism at all for their primary notification path. When a signal is read inside a template, Angular's reactive graph registers that template's render function as a "consumer" of that specific signal (a producer/consumer graph, similar in spirit to fine-grained reactive libraries like SolidJS or Vue's Composition API). When the signal changes (subject to its own `equal` function, `Object.is` by default), only the consumers that actually read it get scheduled for a targeted refresh — independent of whether a parent passed down a "new" object reference through `@Input()`. This means a signal can be read across component boundaries (e.g., injected from a service) and cause precise updates without any component needing new input references or explicit `markForCheck()` calls at all.
Interviewer intent: Distinguishes candidates who've only used signals superficially from those who understand *why* signals plus OnPush (or zoneless mode) is architecturally different from the classic input-reference dance.

Q7. Why does OnPush still work correctly when an event fires inside the component itself, even without an input change?
A: Because Trigger 2 is independent of Trigger 1. Any DOM event bound in the component's own template (click, input, etc.) causes Angular (via zone.js patching `addEventListener`) to run a change-detection pass that includes the path from the event's originating component up to the root. The originating component is therefore always checked on its own local events, regardless of whether any of its inputs changed — this is why simple counters and toggles work "for free" under OnPush without any manual `markForCheck()`.

Q8. What's the difference between `ChangeDetectorRef.detectChanges()` and `markForCheck()`, and when would misusing one hurt OnPush's benefits?
A: `detectChanges()` synchronously runs change detection immediately for this view and its descendants, ignoring dirty flags — it forces a check right now. `markForCheck()` merely flags the component (and escalates dirty-path marking up to the root) so it will be included in the *next* scheduled tick, without forcing an immediate synchronous run. Calling `detectChanges()` repeatedly in a hot path (e.g., a `mousemove` handler) effectively reintroduces Default-strategy-like unconditional checking for that subtree on every event, defeating the purpose of OnPush and potentially causing performance regressions worse than not using OnPush at all, since you've now added the overhead of manual invocation on top.

Q9. A component displays `user.address.city`, bound via `[user]="user"` where `user` is an OnPush `@Input()`. A service mutates `user.address.city` directly. What happens, and how do you fix it?
A: Nothing renders, because the top-level `user` object reference is unchanged — the `Object.is` comparison on the `@Input()` binding sees no difference and never marks the component dirty, even though the underlying data did change. The template may appear correct only if some *unrelated* trigger elsewhere causes a check to run after the mutation, which is misleading and not something to rely on. The fix: the service (or whichever code owns the mutation) must produce a new object at every level on the path from the root object down to the changed field — e.g., `user = { ...user, address: { ...user.address, city } }` — so the reference passed to the `@Input()` actually changes.

Q10. How would you migrate a large Default-strategy application to OnPush incrementally without breaking things?
A: Do it leaf-first, one component subtree at a time, not globally. (1) Pick a leaf component with no children (or few), add `changeDetection: OnPush`. (2) Audit every mutation of data reachable from its `@Input()`s and replace in-place mutations with immutable updates (spread, `map`/`filter`, Immer, or `structuredClone`). (3) Run the app / relevant tests and use Angular DevTools' change-detection highlighting (or a manual console-log in `ngDoCheck`) to confirm the component still updates when it legitimately should and stops updating on unrelated ancestor events. (4) Move up the tree to the leaf's parent once all its children are OnPush-safe, repeating the audit for data the parent owns and passes down. (5) For state genuinely shared across sibling subtrees, migrate the backing store to expose `Observable`s (consumed via `| async`) or Signals rather than raw mutable objects, since that removes the reference-discipline burden entirely going forward. (6) Only flip `ChangeDetectionStrategy.OnPush` at the `AppComponent`/module root once every subtree underneath has been converted and verified, since a stray Default-strategy component deep in the tree can mask bugs in a partially-migrated ancestor by forcing coincidental full re-checks.
Interviewer intent: Looking for a concrete, risk-aware rollout plan (leaf-first, verify-as-you-go) rather than "just add OnPush everywhere and fix bugs as they come up," which is how migrations go wrong in real codebases.

Q11. Does `ngOnChanges` still fire on an OnPush component even if the template doesn't re-render?
A: `ngOnChanges` fires whenever an `@Input()` reference actually changes (Trigger 1) — that's the same condition that also marks the component dirty for template re-checking. So in practice, if `ngOnChanges` fires, the component *will* also be checked on the next tick; you cannot get the lifecycle hook without the corresponding dirty-marking, because they're driven by the same underlying reference-comparison mechanism (`bindingUpdated`).

Q12. You unit test an OnPush component: you set `component.items = [...newItems]` directly on the instance (not through a parent template) and expect the DOM to update after calling `fixture.detectChanges()` once at the start of the test. It doesn't. Why, and how do you fix it?
A: The single `fixture.detectChanges()` call at test setup ran the *initial* check (which also runs `ngOnInit`),

consuming the "component needs checking" opportunity once. Setting `component.items` afterward directly mutates the field but does not go through the `@Input()` binding-evaluation path (there's no parent template re-executing `[items]="..."` expressions in a shallow test), so none of the four triggers fire, and the dirty flag is never re-set. Calling `fixture.detectChanges()` again after the assignment forces a fresh, unconditional check of that fixture's root view (bypassing the OnPush dirty check for the root under test, similar to `detectChanges()` on a `ChangeDetectorRef`), which correctly re-renders the template with the new value. The fix is simply: call `fixture.detectChanges()` again after any test-level mutation of bound properties.
Interviewer intent: Confirms the candidate knows `fixture.detectChanges()` isn't "check once and stay reactive forever" — a frequent source of flaky/misunderstood OnPush unit tests.

Q13. Why is `Object.is`-based reference equality preferred over deep equality checks for OnPush's input comparison, from a performance standpoint?
A: Deep equality (recursively walking objects/arrays to compare contents) is O(n) in the size of the data structure and must run on every change-detection cycle for every bound input on every OnPush component — for large or deeply nested objects this cost can dominate or exceed the cost of just re-rendering unconditionally, defeating the purpose of skipping work. Reference comparison is O(1) regardless of data size. The tradeoff Angular makes is to push the O(n) cost (of producing a new reference only when data truly changes) onto the write path, which happens far less often than the read/check path in typical UIs, and to make that cost explicit and controllable by the developer rather than implicit and unavoidable inside the framework's hot loop.

Q14. Can two sibling OnPush components sharing a mutable object via a common service ever get out of sync, and how do you prevent it structurally?
A: Yes — if both siblings hold `@Input()`-style or field references to the *same mutable object* from a shared service, and one component mutates a nested property directly, neither sibling's OnPush check is triggered (no `@Input()` reference change occurred, no event necessarily happened in the other sibling, no async pipe involved), so the other sibling can silently show stale data even though both "point to" logically live state. Structurally, prevent this by never sharing raw mutable objects across component boundaries for state that must synchronize visually: use a `BehaviorSubject`/signal in the service as the single source of truth, have each component consume it via `| async` or a signal read, and always replace (not mutate) values when writing to the store. This guarantees every consumer is notified through one of the four legitimate triggers rather than relying on coincidental re-renders elsewhere in the tree.

Q15. What's a scenario where you would deliberately use `ChangeDetectorRef.detach()` alongside OnPush, and how does it differ from OnPush alone?
A: A live dashboard widget receiving very high-frequency updates (e.g., 60 times/second market ticker) where even OnPush's per-tick dirty-check overhead across the rest of a large tree is unwanted for that specific subtree. Calling `detach()` removes the component's view entirely from Angular's automatic tick cycle — it will not be checked even if marked dirty by any of the four triggers — until you manually call `detectChanges()` (or `reattach()`) yourself, typically throttled via `requestAnimationFrame` or `setInterval`. This differs from OnPush alone, which still participates in the normal tick cycle and will be checked whenever legitimately marked dirty; `detach()` gives you full manual control over *if and when* checks happen at all, independent of the strategy, and is a stronger, more surgical tool reserved for genuinely hot paths rather than general-purpose usage.

7. Quick Revision Cheat Sheet

- **OnPush checks a component only when:** (1) an `@Input()` gets a new reference (`Object.is`

comparison), (2) a DOM event fires inside the component's own template/subtree, (3) `AsyncPipe` emits and calls `markForCheck()`, (4) explicit `markForCheck()` or a changed signal read in the template.

- **Comparison is reference equality only** — never deep/structural. Primitives compare by value naturally; objects/arrays must be replaced, not mutated.
- **Immutable patterns:** spread (`{...obj}`, `[...arr]`), `structuredClone()`, `array.map/filter` instead of `push/splice`, Immer's `produce()` for deep nested state, `Object.freeze()` to catch accidental mutation in dev.
- `markForCheck()` flags the component and escalates up the ancestor chain so the walk isn't pruned before reaching it, scheduled for the *next* tick. `detectChanges()` forces an immediate, synchronous check of this view + descendants right now, bypassing dirty flags — overuse defeats OnPush's purpose.
- **AsyncPipe = free** `markForCheck()` on every emission — the canonical OnPush companion for `Observable`/`Promise` bindings.
- **Signals bypass reference-equality entirely**: fine-grained producer/consumer graph triggers exactly the templates that read a changed signal, with no `@Input()` plumbing or manual CD calls needed — this is also what makes zoneless Angular viable.
- **Local mutation inside the component's own event handler still works** — Trigger 2 checks the component itself regardless of input changes; the risk is only for *other* components (siblings/ancestors not on the event path) relying on the same mutated object.
- **Deep/nested mutation without replacing the top-level reference is invisible** to OnPush — the #1 real-world bug source. Third-party DOM/data mutation is equally invisible since it never touches Angular-owned state.
- **Testing:** `fixture.detectChanges()` must be called again after any test-level mutation of a bound `@Input()`/field — it does not stay "live" after the first call, especially for OnPush components in shallow (non-parent-driven) test setups.
- **Migration strategy:** leaf-first, audit mutations at each level, verify with DevTools' CD profiler, move upward; convert shared mutable state to `Observable`/signal-backed stores before flipping ancestors to OnPush.

Created By - Durgesh Singh